Web-Based Village Pond Planning System

CSD Assignment 1 – Phase 3 Final Technical Report

SNEHA NAGMOTI, Indian Institute of Technology Bhilai, India

This report presents a web-based decision-support system for preliminary village pond planning. A user can select a study centre and radius on an interactive satellite map, or upload surveyed contour geometry in KML/KMZ form. The backend reconstructs a terrain surface, removes artificial depressions with Priority-Flood, resolves flats, computes D8 flow direction and accumulation, delineates the contributing watershed, and ranks spatially separated pond candidates. Mapped OpenStreetMap hydrography, satellite colour screening and a modelled major-drainage corridor, each with a 60 m buffer, exclude rivers and surface water before ranking; colour screening also identifies candidate open land. Historical rainfall is aggregated from complete calendar years; annual runoff is estimated by $V = C(P/1000)A$, with rainfall P in millimetres and area A in square metres. A geometry model converts usable runoff into pond depth, capacity, footprint and excavation dimensions while enforcing side-slope, freeboard and land-area limits. Results are returned as structured JSON and visualized as map overlays. In the verified live example (15 October 2026), the system reported a 72.73 ha catchment, 1,324.2 mm annual rainfall, 288,908 m^3/year runoff and one eligible site; the selected design stores 231,126 m^3. The delivered implementation includes a responsive React/Leaflet interface, a documented FastAPI service, public-data fallbacks, validation and rate limits, and automated backend and frontend tests.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

1 Introduction

Seasonal rainfall can provide substantial local storage, but selecting a pond site is difficult because elevation, drainage, land cover and rainfall must be interpreted together. A site chosen only from a visual map can fall on a ridge, intercept little contributing area, overlap water, or require impractical excavation. The project therefore implements a geospatial web application that turns a selected land area or uploaded contour map into auditable pond-planning evidence. It produces a suggested site, watershed boundary, rainfall statistic, expected runoff, pond volume and dimensions, and renders these results directly on the map. The following sections define the requirements, architecture, hydrological method, implementation, CSD decisions and measured results.

1.1 Motivation

Manual screening is slow and inconsistent: contour interpretation is specialist work, rainfall records are distributed across services, and current land or surface-water conditions are not obvious from coordinates alone. Combining these inputs in one repeatable pipeline helps an administrator compare alternatives before commissioning a field survey. The tool is deliberately a screening system: it makes the data and assumptions visible instead of replacing engineering approval.

Author's Contact Information: Sneha Nagmoti, Indian Institute of Technology Bhilai, Durg, India, snehan@iitbhilai.ac.in.

1.2 Scope of the Project

The system (i) supports map-based centre/radius selection and KML/KMZ contour upload; (ii) reconstructs terrain and drainage; (iii) identifies up to three ranked, separated pond options outside detected water; (iv) estimates catchment area, annual rainfall, runoff, storage capacity and excavation geometry; and (v) visualizes the watershed, candidates, elevation and drainage evidence. It does not certify land ownership, soil permeability, dam stability, flood safety or construction drawings. Final selection therefore requires cadastral, hydrographic, geotechnical and field verification.

2 Problem Statement and Requirements

The required system must accept a user-defined land area, analyze its terrain and rainfall, recommend a pond site, and display the site, catchment and expected water volume on a usable frontend. Table 1 maps each requirement to the implemented component.

Table 1. Functional requirements and implementation

Requirement	Implemented in
Satellite map and land selection	React/Leaflet map, search, coordinate and radius controls
Contour visualization/upload	Upload workspace and `/api/analyze-contour`
Candidate-land identification	Mapped hydrography, RGB/HSV satellite screening and buffered major-drainage exclusion
Catchment estimation	Priority-Flood, resolved-flat D8 and reverse watershed traversal
Historical rainfall	Open-Meteo archive with NASA POWER fallback
Runoff and pond sizing	`terrain.py` volumetric runoff and trapezoidal geometry
Combined result view	Toggleable Leaflet layers, ranked markers and result cards
Structured API output	Validated Pydantic response models and Swagger/ReDoc

2.1 Non-Functional Requirements

The application is responsive on current desktop and mobile browsers and keeps analysis work asynchronous so external calls do not freeze the interface. The API validates coordinate, radius, upload type, archive contents and size; applies per-route rate limits; restricts trusted hosts/CORS; and returns explicit source-quality metadata. The backend is stateless when optional history is disabled, so identical replicas can be placed on the four allotted systems. External failures use bounded timeouts, retries, cache and documented fallbacks rather than fabricated values. The production frontend is a static bundle, while health endpoints support availability checks. Target interaction latency is seconds for live screening and under one minute for a 15 MiB contour upload.

3 System Architecture and High-Level Design

The browser collects a location/radius or multipart contour file and calls the FastAPI service over HTTP(S). Independent asynchronous adapters obtain elevation, satellite imagery, mapped hydrography and historical rainfall. The terrain and vision services normalize those inputs, the hydrology service computes drainage and candidates, and a sizing service produces runoff and pond geometry. Typed response models return both values and map geometry. Optional PostgreSQL history is isolated from analysis and is disabled in the public demo; cache entries reduce repeated public-API calls. Figure 1 shows this flow.

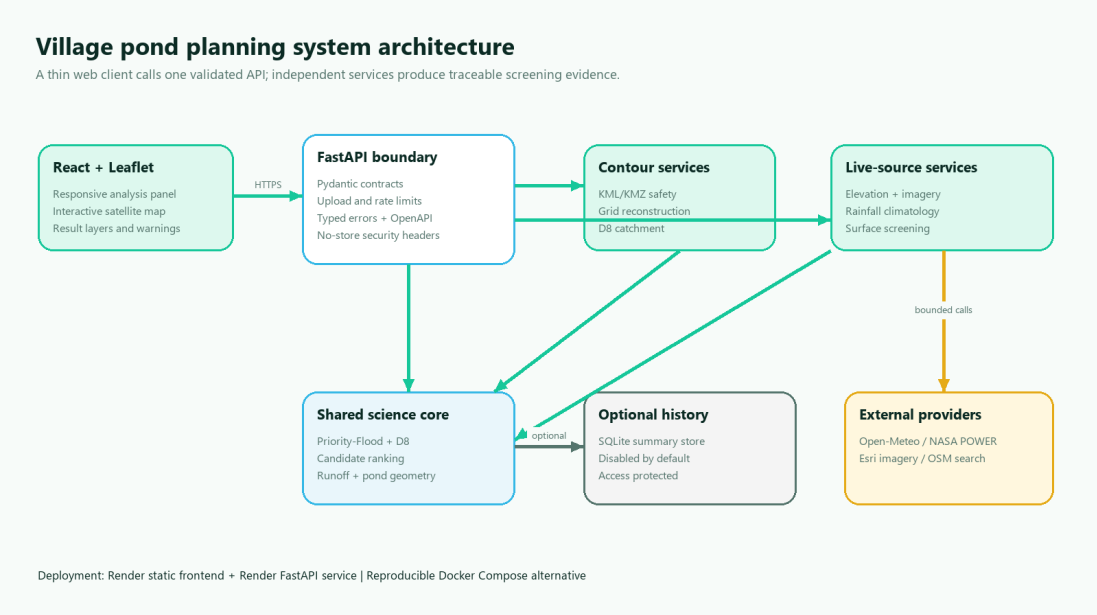


Fig. 1. High-level client, API, analysis-service and public-data architecture.

3.1 Technology Stack

The frontend uses React, Vite and Leaflet. The backend uses Python, FastAPI, Pydantic, NumPy, OpenCV and HTTPX. Elevation comes from Open-Meteo/Copernicus GLO-90 compatible data with a bounded tile fallback; rainfall uses Open-Meteo Archive and NASA POWER; Esri World Imagery supplies the displayed and analyzed satellite mosaic, and OpenStreetMap map tiles supply mapped rivers and water bodies. SQLAlchemy with PostgreSQL provides optional history. Pytest and Vitest test the two layers, and Docker/Render configurations provide reproducible deployment. This stack was chosen because its geospatial libraries implement the required raster/vector operations while FastAPI exposes an inspectable JSON contract.

4 Methodology

Both input modes enter the same hydrological core. Live mode samples an elevation grid over the chosen radius. Contour mode parses KML/KMZ safely, extracts line elevations, projects coordinates to metric space,

rasterizes observations and performs fixed-observation harmonic interpolation. The latter iteratively replaces unobserved cells by neighboring means while never changing contour cells; convergence and observed-cell ratio are reported in the response.

4.1 Terrain and Elevation Analysis

The DEM is depression-filled with Priority-Flood so isolated numerical pits do not trap all flow. A small deterministic gradient resolves equal-height flats. Each valid cell then selects its steepest lower neighbour among the eight adjacent cells (D8). Processing cells in topological order gives flow accumulation in $O(n)$ after the $O(n \log n)$ flood, where n is the number of grid cells. Contours displayed by the frontend are regenerated from this surface. For uploaded data the original contour lines remain visible, making interpolation and source coverage auditable.

4.2 Catchment Area Delineation and Site Ranking

Candidate cells must lie inside the study/candidate-land mask, away from the analysis boundary, on a routable cell, and outside three buffered exclusions: OpenStreetMap-mapped water, satellite-classified water, and a modelled major-drainage corridor (cells draining at least 8% of the grid). Water masks are aggregated over each terrain cell's footprint, so a narrow mapped stream cannot vanish between grid centres. Automatic, point and region selection all apply these exclusions before ranking; a clicked point inside them is rejected with a reason. A weighted score favors high log-flow accumulation (0.52–0.58 weight), low elevation, low local slope, interior clearance and distance from detected water. Non-maximum spatial separation prevents the top three options from collapsing onto the same point. For a chosen candidate, reverse traversal follows every D8 link that drains to it; the number of upstream cells multiplied by projected cell area gives the contributing area. Thus the reported catchment belongs to the selected pond point rather than to an unrelated study-area outlet. The drainage path is traced downstream for visual inspection.

4.3 Rainfall, Runoff and Pond Geometry

Daily precipitation is grouped into complete calendar years; incomplete years are excluded before computing mean annual rainfall. If the primary archive is unavailable, NASA POWER is queried with the same completeness checks. Annual runoff is

$$V = C \left(\frac{P}{1000} \right) A, \tag{1}$$

where A is catchment area in m^2, P is annual rainfall in mm, and the course-demo coefficient is $C = 0.30$. The coefficient and its source are returned explicitly. A configurable capture fraction converts runoff to target storage. Pond length, width and water depth are solved for a rectangular basin with 2H:1V side slopes and 0.5 m freeboard, using the exact sloped-basin volume $V_p = WLd + s(W + L)d^2 + \frac{4}{3}s^2d^3$; dimensions are reduced if the detected candidate land cannot contain the excavation footprint.

5 Implementation

5.1 Backend and API Design

The main routes are summarized in Table 2. The upload route accepts either `contour_file` or the assignment-compatible `contour_map` field; `/api/analyzeContour` and `/api/findCatchment` remain compatibility aliases.

Validation errors use 4xx responses; third-party failures are bounded, recorded per source, and either fall back or produce an incomplete result instead of a false success. No login is needed for the public screening demo; history, when enabled, requires its configured access control.

Table 2. Public API endpoints

Method	Path	Purpose
POST	`/api/analyze`	Location/radius analysis
POST	`/api/analyze-contour`	KML/KMZ terrain analysis
GET	`/api/search-village`	Submitted place search
GET	`/health/live`, `/health/ready`	Service checks
GET	`/docs`, `/redoc`	API documentation

5.2 Frontend and Visualization

The interface offers Live Analysis and Contour Upload workspaces. A collapsible result panel leaves the map available for inspection. Users can select a location by search, coordinates or map interaction, set the radius, upload KML/KMZ, choose automatic/point/region contour selection, and switch among ranked options. Leaflet overlays show the study radius, source contours, reconstructed elevation surface, candidate land, water exclusion, catchment boundary, drainage path and numbered pond sites. Result cards present catchment, rainfall, runoff, capacity, depth, dimensions, score and source status in plain language. Figure 2 shows the deployed result panel with the river and water safeguard and the ranked options.

5.3 Database and Storage

Analysis responses are self-contained and do not depend on persistence. An optional PostgreSQL store can retain selected summary records for a protected history view, but public deployment disables history to minimize stored user data. Public-source responses use bounded local caches with expiry; uploaded archives are processed in memory/temporary storage and are not used as permanent data.

6 CSD Themes and Topics Applied in the Project

The most important decision was a modular monolith rather than premature microservices: geospatial arrays remain in one process, avoiding serialization overhead, while source clients and algorithms still have clean interfaces. Asynchronous public-data calls shorten the critical path, while CPU-heavy terrain work remains deterministic and testable. Quality metadata is treated as part of the API contract; a successful HTTP call cannot silently turn missing rainfall or land evidence into a precise engineering answer. Because analysis is stateless, the same container can be replicated across the four laboratory systems and placed behind a reverse proxy if concurrency requires it.

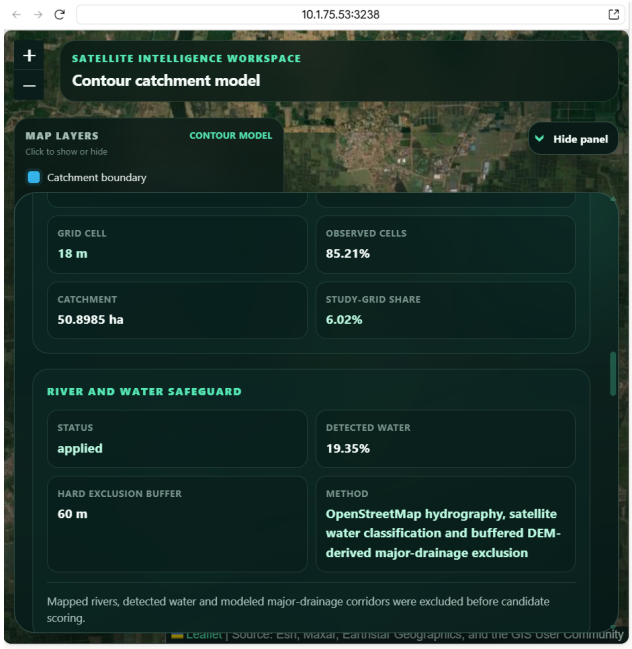


Fig. 2. Deployed result panel (15 October 2026): water safeguard applied with a 60 m buffer, and the ranked, spatially separated pond options with their upstream area and water/channel clearance.

7 Results and Evaluation

The deployed live workflow was re-verified on 15 October 2026 at 21.244025°N, 81.288000°E (2 km radius). After water and drainage exclusion one eligible site remained, giving the values in Table 4; the status was *degraded* because the elevation and rainfall fallbacks were used, and the interface reports this. The selected design stores 80% of annual modeled runoff and respects the configured depth, side-slope and footprint constraints. The supplied `contours_1m.kml` was also processed generically: 1,355 contours and 159,113 source points formed a 148×181 grid with 18 m cells; 85.206% of grid cells were directly observed and harmonic interpolation converged in 31 iterations. With water exclusion applied, the automatic site had a 50.90 ha catchment, 195,470 m^3/year runoff and 156,376 m^3 storage; re-selecting each of the three displayed options returned a valid catchment (22.23–63.24 ha). Its coloured DEM and ranked sites are shown in Figure 3. These are model-derived screening results, so correctness is evaluated by invariants, source traceability and map coherence rather than unavailable field ground truth.

7.1 Performance and Verification

Tests cover Priority-Flood/D8 behavior, independent reference-walk checks of flow accumulation and catchment membership, point/region selection, water exclusion, contour parsing, upload limits, source failures, runoff/geometry constraints and map-result presentation. On 15 October, measured from a laptop on the college network, the supplied KML returned in 6.2 s through the college HTTP endpoint, repeated point and region requests in 2.0–2.2 s, and a live request in 3.2 s; these timings benefit from warm tile caches

Table 3. CSD concepts and concrete use

Theme	Application and rationale
REST contracts	Versionable typed requests/responses separate UI from analysis.
Concurrency	Elevation, imagery, hydrography and rainfall are awaited concurrently to reduce wall time.
Caching	Repeated public-data queries reuse bounded TTL entries, reducing latency and quotas.
Modularity	Source adapters and hydrology/vision services can be replaced independently.
Resilience	Timeouts, retries, fallbacks and quality states expose partial-source failures.
Security	Upload/archive validation, limits, CORS, trusted-host and rate controls reduce attack surface.
Scalability	Stateless analysis and a static frontend allow independent replication on allotted hosts.
Testing/CI	Unit, integration and UI tests verify algorithms, contracts and Nginx security headers.

Table 4. Representative live-analysis result

Measure	Result
Catchment area	72.73 ha
Mean annual rainfall	1,324.2 mm/year
Expected runoff ($C = 0.30$)	288,908 m^3/year
Suggested pond location	21.248925, 81.300267
Suitability score	97.0 / 100
Water storage capacity	231,126 m^3
Water/excavation depth	4.0 / 4.5 m
Water dimensions	300.35 × 205.57 m
Excavation footprint	62,759 m^2

on the server. These are functional and numerical-consistency checks, not validation against a surveyed watershed.

On the allotted container's 512 MiB memory and one-CPU quota (read from its cgroup limits), two simultaneous supplied-KML uploads returned HTTP 200 in 5.77 s and 5.91 s with results identical to the sequential run, while ten readiness probes all succeeded (maximum 0.354 s). An independent multiplication of area, rainfall and coefficient agreed with every reported runoff volume. This bounded concurrency test supports a small demonstration workload; it is not a sustained saturation benchmark or a high-availability claim.

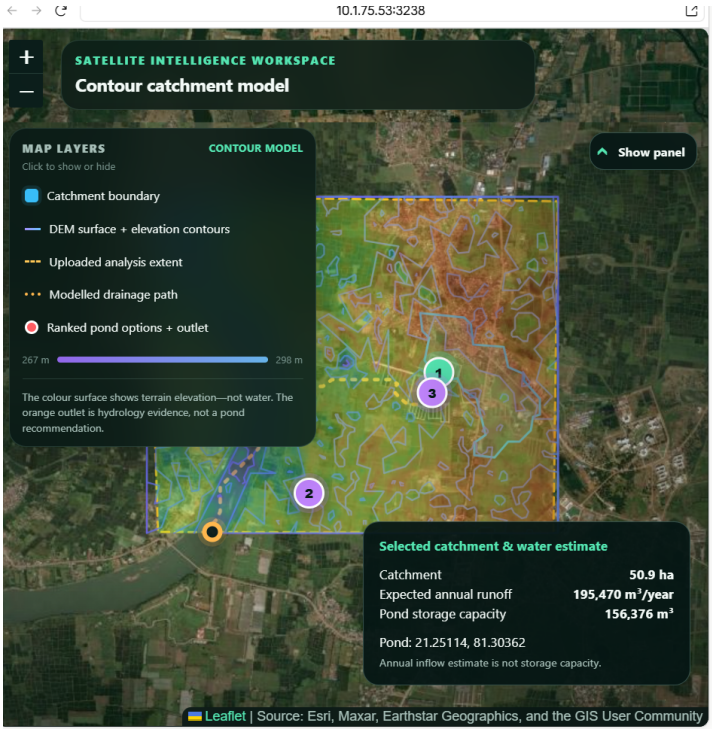


Fig. 3. Allotted-system contour result for `contours_1m.kml` (15 October 2026): coloured DEM, catchment, drainage path to the outlet and three ranked options on land east of the river, with the selected pond's 50.9 ha catchment, 195,470 m^3/year runoff and 156,376 m^3 storage shown on the map.

Final checks (15 October 2026). All 85 backend tests and 14 frontend tests, both linters, the production build and the GitHub CI workflow passed for commit `c641fc2`. The files deployed on system 2 were checked against that commit with a SHA-256 manifest. Through the college URL, each displayed contour option and a drawn region returned valid catchments, runoff equalled $C \cdot A \cdot P$, storage was at most 80% of runoff, and missing uploads, wrong field names and malformed KML returned HTTP 422. Earlier browser checks covered coordinate validation, layer/panel toggles and a 390-pixel layout without horizontal overflow. Public data sources can still fail; such failures are reported as errors or incomplete results, not hidden.

7.2 Allotted-System Deployment and Recovery

The frontend and API run together on allotted system 2, accessed through SSH port 2238. External HTTP port 3238 forwards to its internal port 3000. The frontend uses relative `/api` requests, so the demonstration does not depend on the Render backend. User-owned Supervisor restarts an exited application process; this recovery was verified. A container reboot requires the documented start command because the container's initial process is SSH rather than a service manager; on 15 October the container had restarted, and the service was started with that command before re-verification. Successful satellite tiles are cached with a bounded lifetime, allowing missing-tile retries without accepting partial imagery. Intermittent college-network and upstream connection failures remain possible; source failures are shown explicitly.

The map displays the selected catchment boundary and pond options, with an on-map summary of catchment area, expected annual runoff, coordinates and storage capacity. Annual collected runoff and physical storage capacity are labelled separately. On narrow screens the summary remains visible with the controls panel collapsed. Deployment commands, resource checks and reproducible verification scripts are included in the repository.

8 Discussion and Limitations

The strongest result is traceability: every recommendation is accompanied by its watershed, ranked alternatives, source states and assumptions. Nevertheless, GLO-90-scale/public elevation cannot resolve small bunds and culverts; harmonic reconstruction between contours smooths unmeasured terrain; mapped hydrography plus RGB/HSV screening can still miss unmapped seasonal or recently changed channels; a uniform runoff coefficient ignores soil and land-use variation; and long-term annual rainfall does not replace design-storm routing. Land ownership is not inferred. The map therefore supports comparative screening, not construction approval. Future work should incorporate verified cadastral parcels, higher-resolution survey DEMs, official hydrography, soil-specific coefficients, IDF curves, spillway routing and measured validation sites.

9 Submission Links

Table 5. Clickable submission and verification links

Item	Link
GitHub repository	Open source repository
Public demo video	Watch the five-minute demonstration
Public frontend	Open public web application
Allotted-system frontend	Open college-system application
Interactive API docs	Open Swagger documentation
Contour-analysis API	POST /api/analyze-contour (field: `contour_map`)

The allotted-system URLs require access to the college network. The public demonstration video and Render frontend remain accessible outside that network.

10 AI-Assistance Disclosure

AI tools were used for reviewing code and for report formatting. All technical statements were checked against the source code, automated test results, deployed API responses and map output. AI output was not used as hydrological ground truth; terrain, catchment and runoff results are produced by the implemented algorithms from the stated inputs and assumptions.

References

[1] Open-Meteo, “Historical Weather and Elevation APIs,” https://open-meteo.com/.
[2] NASA POWER Project, “Daily Meteorology API,” https://power.larc.nasa.gov/.

[3] Copernicus Programme, “Copernicus DEM GLO-90,” European Space Agency.
[4] Leaflet, “An open-source JavaScript library for mobile-friendly interactive maps,” https://leafletjs.com/.
[5] FastAPI, “FastAPI framework documentation,” https://fastapi.tiangolo.com/.
[6] R. Barnes, C. Lehman, and D. Mulla, “Priority-Flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models,” *Computers & Geosciences*, vol. 62, pp. 117–127, 2014.